Introduction

Purpose:
In this lab you will learn how to use the encoder and run the motor. The encoder is a motor position sensor. It has more complex inputs/outputs than the sensors we covered in the previous lab session. You will also wire your motor and H-bridge to the Arduino microcontroller board and to an external power supply. Lastly, you will find the torque needed to overcome friction.

Safety:

- In your previous lab session, all of the sensors could be powered directly from the Arduino because they used a very low current. For the motor to have enough torque and speed, we need to apply voltages up to 9-12V and currents much larger than Arduino can output, so we need to use an external power supply. This can supply enough current to injure you**. Make sure the power supply is unplugged and off whenever you are connecting any wires.**
- **Make sure someone else knows where you are or that you have a buddy when turning on the power for the first time.**
- **Before turning on the power supply, make sure that everyone is out of the way of any moving components, and the moving components are not in a position to run into any other objects.**
- When testing out your system, do not wear long sleeves, jewelry, or loose-fitting clothing.
- Long hair should be pinned up.
- **Be aware that if the power to the motor or controller is shut off and then restored, the controller will continue to run the program it was previously running.**
- Do not touch the spinning shaft of the motor.
- When testing out your motor, always start with the voltage at the lowest possible level, and gradually increase it until you can get the motor to move. Do not increase the voltage above 10V.

Parts you will need:

- Your linkage mechanism with motor and transmission installed
- The assembly from your previous lab session, including:
- Snap Action Switches (a.k.a. limit switch, micro switch)
    - Toggle Switch
    - Limit Switch
- 400-point Breadboard
- Arduino Uno microcontroller board
- H-bridge Motor Driver (L298 Motor Driver)
- Power supply (provided to you at the beginning of the lab).

The following parts will be on the lab table in front of you. **You must keep them in the mechatronics lab since they will be used by the next ME350 section:**

- Multimeter
- Wire strippers/pliers
- Phillips-head screwdriver

Power supply

Setup:

1. Place your mechanism on the playing field. Have the linkage at the starting position.
2. If any wires are disconnected, reconnect all of the wires from all of the exercises in the previous lab session. Refer to your wiring diagram.
3. Run your sketch for the proximity sensor and open the serial monitor to verify that the wiring was connected correctly.
4. Run your sketch for the limit switches and open the serial monitor to verify that the wiring was connected correctly.
5. Run your sketch for the toggle switch and open the serial monitor to verify that the wiring was connected correctly.

Exercise 1: Encoder

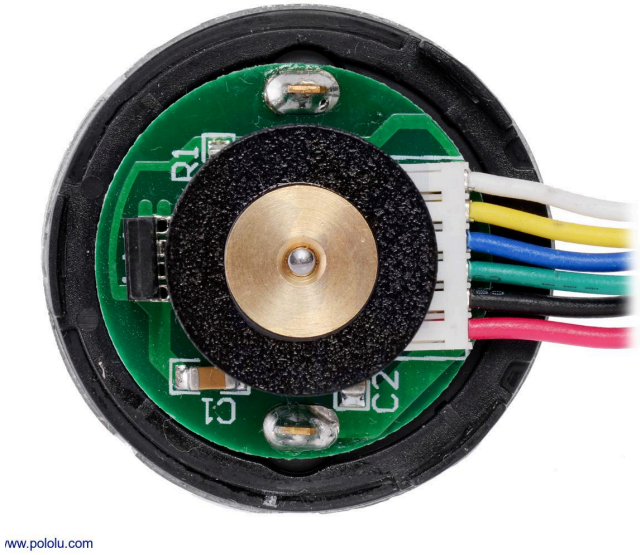


Figure 1: Hall Effect Encoder

Your motor contains a Hall Effect encoder which has two channels, A and B. This type of sensor is similar to an optical encoder, but uses a magnetic field and a wire instead of an LED and photodiode. An encoder is a sensor which provides a digital output as a result of a linear or angular displacement. Our encoder is rotary and will allow us to measure angular displacement.

The encoder has two channels, A and B. The square wave signals, called pulse trains, from the A and B channels are shown in Figure 2 below. Notice that the waveforms are 90 degrees out of phase. By watching which channel rises first, the controller can tell whether the shaft is turning clockwise or counterclockwise.

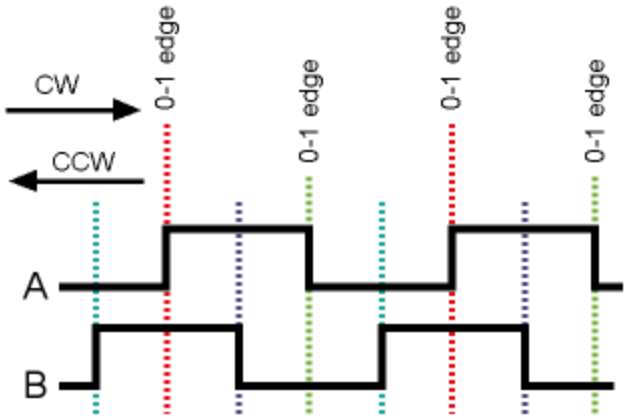


Figure 2: Square waveform signal from two channels of encoder (www.arduino.cc)

The motor in your project contains a hall-effect encoder attached to the motor shaft. This means that the encoder will read **revolutions of the motor**, not revolutions of the output shaft of the gearbox.

Info on this motor and encoder can be found by searching on the Pololu for the project kit part number (4752).

Exercise 1.1: Wiring the Encoder

While completing today's lab exercises, update your wiring diagram from your previous lab.

Wires needed:
(1) 10"' solid core blue, (1) 10" solid core green, (1) 12" solid core yellow, and (1) 12" white solid core

1. The motor wiring has already been bundled into a female connector. Thus, you don't need to solder the wire end. When you follow each of the steps below, you can plug 22-gauge solid core wires (get these as needed from the wire spools) into the other end of the female connector. We will refer to these 22-gauge wires as "jumper wires". This will provide the extra length you need to go from your motor (installed in your mechanism) to your breadboard circuit.

Figure 3: Motor

2. Refer to the Pololu website www.pololu.com (item #4752) for information on the motor and encoder. Scroll down to find the encoder information and read through it. Find the number of counts per revolution and the color codes for the wires. The pin assignment for the motor is provided in the table below:

Color	Function
Red	motor power (connects to one motor terminal)
Black	motor power (connects to the other motor terminal)
Green	encoder GND
Blue	encoder Vcc (3.5 – 20 V)
Yellow	encoder A output
White	encoder B output

3. Using a 10” blue jumper wire, connect the encoder’s Hall Sensor VCC to the positive bus on the right-hand side of the breadboard. This will allow the Arduino to power the encoder.
4. Using a 10” green jumper wire, connect the encoder’s Hall Sensor GND to the negative bus on the right-hand side of the breadboard.
5. Using a 12” yellow jumper wire, connect the encoder’s A channel (Hall Sensor A output) to Arduino digital pin 2.
6. Using a 12” white jumper wire, connect the encoder’s B channel (Hall Sensor B output) to Arduino digital pin 3.
7. Do not connect the other wires from the motor at this time. They are for the motor itself, and will be used in the next exercise.
8. Make sure that the GND pin on the Arduino is still connected to the negative bus on the breadboard (from the previous lab session).

Exercise 1.2: Encoder Position Sketch

In this sketch, we use an interrupt (triggered whenever the A channel or B channel changes from LOW to HIGH or from HIGH to LOW) to call the *updateMotorPosition* function. The *updateMotorPosition* function records four bits of encoder signal, two from channel A and two from channel B, to determine which direction the encoder is turning, and updates the encoder count. This strategy uses quadrature, thus providing the highest resolution possible.

1. Download the “ME350_W25_Encoder_Test.ino”
2. Find the function that says “**updateMotorPosition**()” in the provided code:

```
void updateMotorPosition() {
  // Bitwise shift left by one bit, to make room for a bit of new data:
  encoderStatus <<= 1;
  // Use a compound bitwise OR operator (|=) to read the A channel of the encoder (pin 2)
  // and put that value into the rightmost bit of encoderStatus:
  encoderStatus |= digitalRead(2);
  // Bitwise shift left by one bit, to make room for a bit of new data:
  encoderStatus <<= 1;
  // Use a compound bitwise OR operator  (|=) to read the B channel of the encoder (pin 3)
  // and put that value into the rightmost bit of encoderStatus:
  encoderStatus |= digitalRead(3);
  // encoderStatus is truncated to only contain the rightmost 4 bits by  using a
  // bitwise AND operator on mstatus and 15(=1111):
  encoderStatus &= 15;
  if (encoderStatus==2 || encoderStatus==4 || encoderStatus==11 || encoderStatus==13) {
    // the encoder status matches a bit pattern that requires counting up by one
    motorPosition++;         // increase the encoder count by one
  }
  else if (encoderStatus == 1 || encoderStatus == 7 || encoderStatus == 8 || encoderStatus == 14) {
    // the encoder status does not match a bit pattern that requires counting up by one.
    // Since this function is only called if something has changed, we have to count downwards
    motorPosition--;         // decrease the encoder count by one
  }
}
//------------------------------ End of function updateMotorPosition() -----------------------------//
```

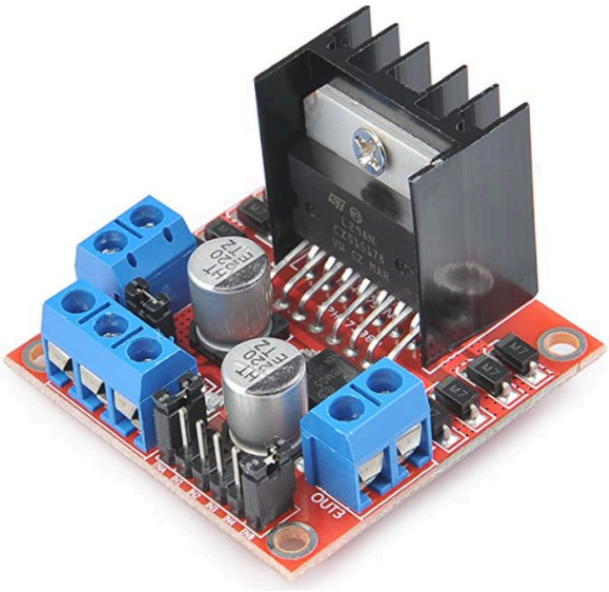
3. Save your sketch and upload it to the Arduino.
4. Open the serial monitor and see the results as you turn your linkage by hand in both directions.
5. Notice that every time you click the limit switch by hand, the motorPosition resets to 0!
6. Start your mechanism at the starting position. Press the limit switch to set the encoder count to zero. **Move the mechanism by hand and record the encoder count at each of the 5 positions listed below**
    a. **TARGET_1_POSITION**: the value of encoder count that corresponds to the target 1 position
    b. **TARGET_2_POSITION**: the value of encoder count that corresponds to the target 2 position
    c. **TARGET_3_POSITION**: the value of encoder count that corresponds to the target 3 position
    d. **TARGET_4_POSITION**: the value of encoder count that corresponds to the target 4 position
    e. WAIT_POSITION: the value of encoder count that corresponds to the waiting position (the position where your linkage waits until a target is detected)

Lab 9 Assignment Part 1

Answer the following questions with your team and record your answers on a doc, then submit to Canvas.

1. How many rotations can this motor perform such that the encoder count doesn't cause overflow of an "int" type variable? How about a "long" type variable? For the motorPosition variable in the Arduino code, would you use an "int" type or a "long" type? Why? Hint: Think about the motor speed and encoder resolution (with quadrature decoding).
2. Why did we make motorPosition a volatile variable? You can find the answer by doing a search for "volatile" on the Arduino website.
3. What does an interrupt do? Search for the answer on the Arduino website.
4. In void setup, the command digitalWrite(PIN_NR_ENCODER_A, HIGH) has the function of turning on a pullup resistor. What is a pullup resistor? Search the internet to find the answer.

Exercise 2: Motor and H-bridge



The H-bridge is a type of motor driver. It allows the voltage to be applied across the motor in either direction. This allows the motor to run forwards and backwards instead of in one direction only. The H-bridge also acts much like a relay, allowing a low voltage signal (from the Arduino) to control the higher voltage and higher current from the power supply. (The Arduino does not supply enough current to power the motor at the torque we need; therefore, we need to use an external power supply.) If interested, here is the chip's datasheet.

The diagram below is a schematic representation of the wiring in a simple H-bridge. It contains four switches. When switches S1 and S4 are closed, the motor will run in the forward direction. When switches S2 and S3 are closed, the current will flow the other way through the motor, causing it to run in the reverse direction. We use the Arduino to apply a HIGH or LOW signal to the I1 and I2 pins on the H-bridge, causing the pairs of switches to open or close.

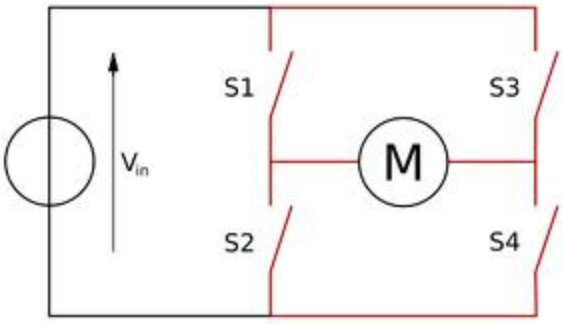


Figure: Diagram of an H-bridge (simplified)

We control the speed of the motor using Pulse Width Modulation (PWM). PWM is simply turning the voltage on and off very quickly for different amounts of time to get a different duty cycle – see the picture below. The Arduino uses the analogWrite command with a value between 0 (0% duty cycle) and 255 (100% duty cycle) to accomplish PWM. Since we will be using 10V from the power supply for your project, then the command

analogWrite(PIN_NR_PWM_OUTPUT, 255) would apply 10V to the motor, and analogWrite(PIN_NR_PWM_OUTPUT, 127) would apply (on average) 5V to the motor.

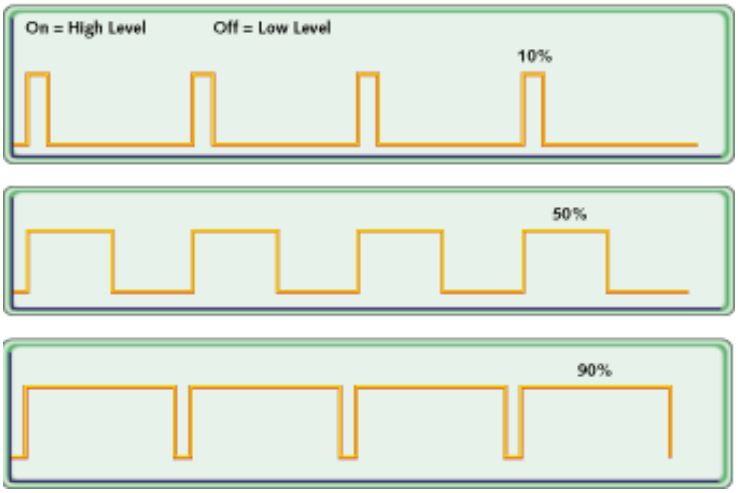


Figure: Pulse Width Modulation

Exercise 3.1: H-bridge wiring

Before powering the motor, we need to test the H-bridge and power supply to ensure they are functional. Throughout this section, we will test both devices by taking measurements with the Digital Multimeter (DMM) in the project kit.

Figure: Digital multimeter.

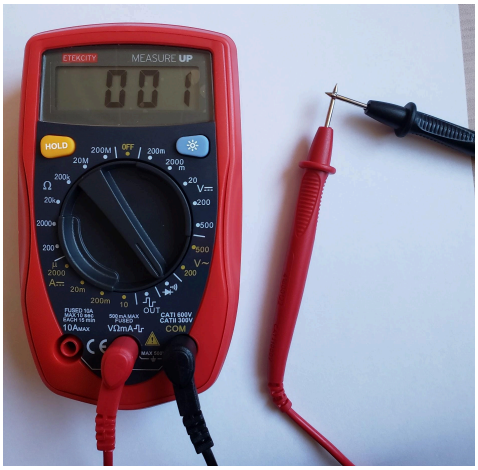
DMMs are an essential tool for electronics. The DMM in our kit can measure resistance, DC and AC voltage, current, and can detect short circuits (shorts). A good practice with DMMs is to turn it off whenever you are not using it.

Safety:

- The tips of the DMM probes are sharp, so please be careful not to poke yourself.
- Do not send more than 500 mA into the center terminal. Knowing how much current will go through the DMM requires an understanding of circuits. If you have questions, please ask your Lab Instructor.

1. Take the DMM probes out of the packaging. Remove the plastic covers on the connectors.

2. Plug the black probe into the right-most terminal labeled COM. Plug the red probe into the center terminal that is labeled VΩmA.
3. We first need to make sure the multimeter itself is working properly. Turn the central dial as shown below with a sound/volume icon. When in this mode, the DMM will beep when the ends of the probes are shorted together. Touch the metal probe tips together. You should hear a loud beep from the DMM.



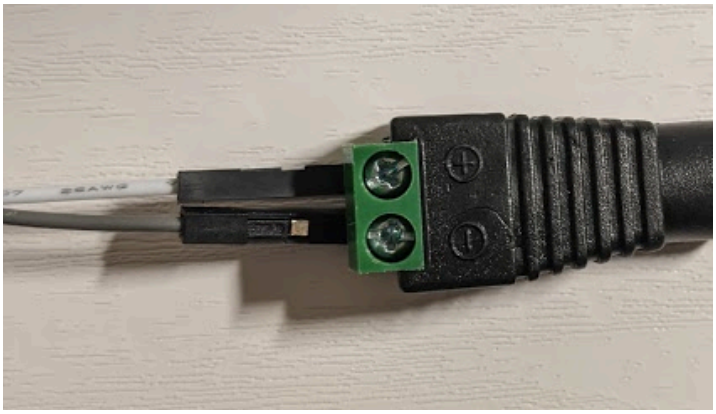
4. Now we will test the power supply with the DMM. Grab the power supply out of your kit. Plug the power supply's jack into the green screw terminal connector.

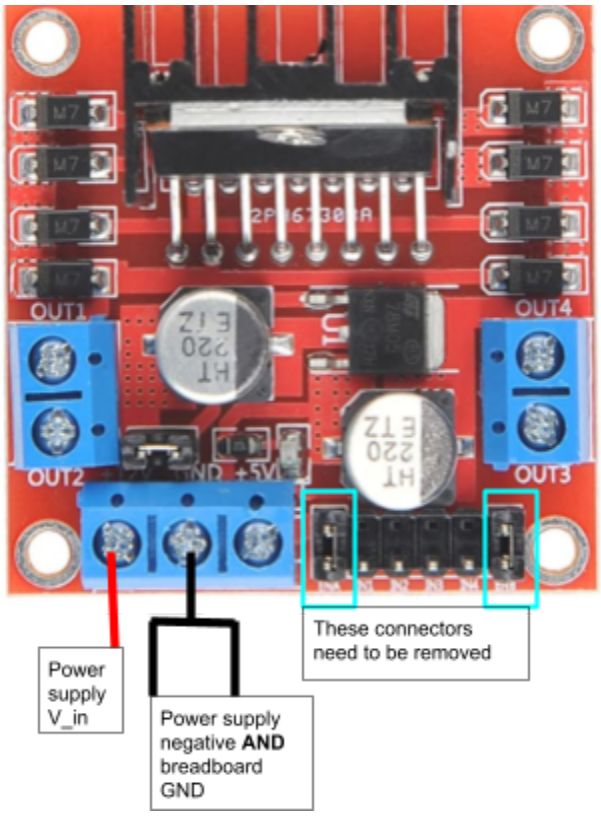
5. Set the voltage on the power supply to **9 Volts** by using the key to turn the dial on the back of the power supply.
6. Plug in the power supply to an outlet.
7. Grab your DMM and set the dial to 20 in the top-right region for DC voltage (meaning it can measure up to 20 VDC). Touch the red probe to the "+" terminal and the black probe to the "-" terminal on the green connector (NOT the screws themselves, but the terminals). You should see a voltage close to 9V on your DMM.
   NOTE: DO NOT let the metal probes touch each other while touching the terminals, since this will short the power supply.

8. **Unplug the power supply.**

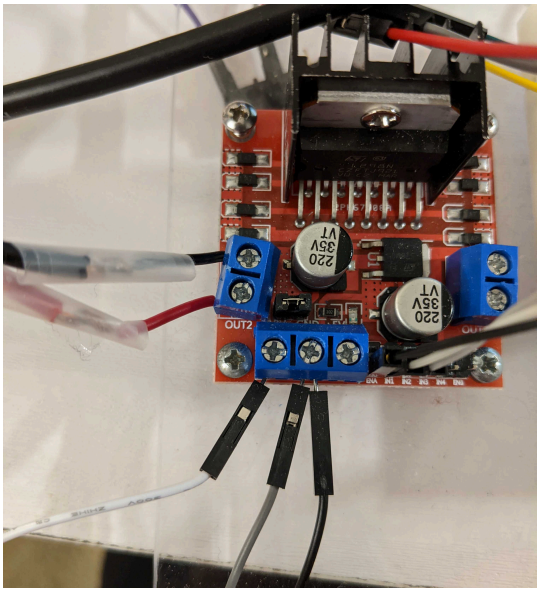
Now you can set your DMM aside. We will use it again shortly.

9. Make sure the power supply is **OFF and unplugged.**
10. Connect the positive terminal on the green screw terminal to the +12V terminal on the H-Bridge.
    a. To use the screw terminals, use a screwdriver to first loosen the screw in the green plastic. Insert the wire into the metal terminal, and then tighten the screw so it clamps down on the wire. Lightly tug the wire to ensure it's clamped. If the wire slides right out, it was not clamped down properly.

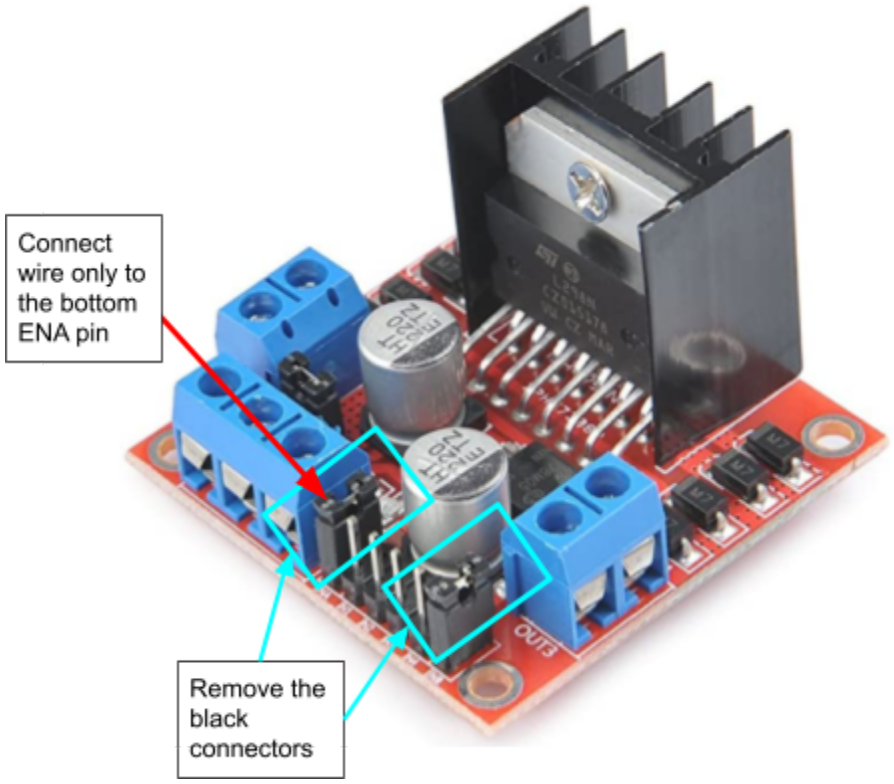




11. Using **two** jumper cables, connect **both** to the GND screw terminal on the H-Bridge.
    a. Connect one of the wires to the negative on the green screw terminal for the power supply
    b. Connect the other to the ground (blue negative rail) on the breadboard
    c. See the image below for an example of what this should look like. The white and gray wires are connected to the positive and negative on the green screw terminals, and the black wire is connected to breadboard ground.
    d. This is important because it is critical to have a common ground between all components. By doing this, we are connecting the power supply ground with the Arduino ground.



12. Connect the **ENA** terminal on the H-bridge to Arduino digital **pin 11**. This pin will be used by the Arduino to send the signal for pulse width modulation to the H-bridge.
    a. There are connectors on top of this pin that **need to be removed** in order to use the PWM signal. You should be able to pull them with your fingers or some pliers.
    NOTE: The connectors are only present for a new H-bridge. If you do not have this connector, continue to step b.
    b. Connect the wire to the pin on the bottom row (closest to the edge of the board), not the top row.

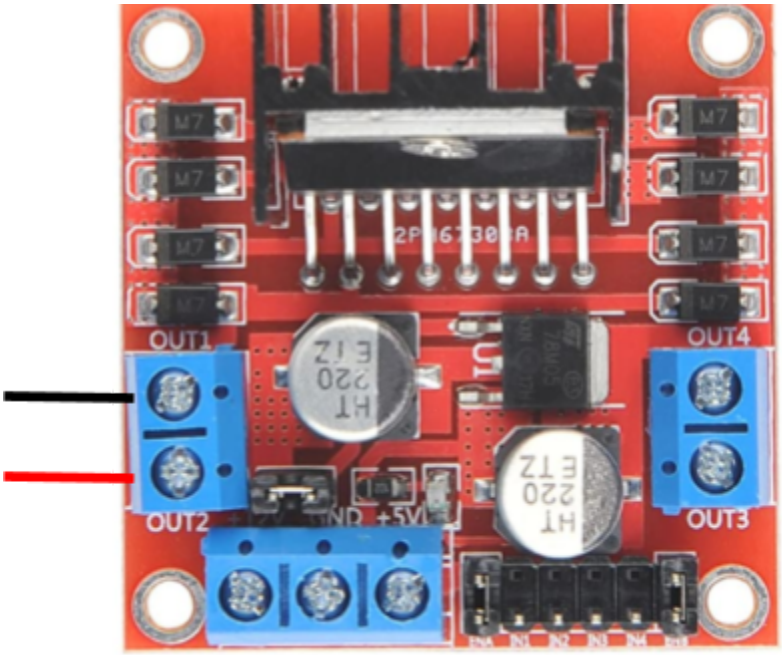


13. Connect the **IN1** pin on the H-bridge to Arduino digital **pin 12**.
14. Connect the **IN2** pin on the H-bridge to Arduino digital **pin 13**. The Arduino will send the signal for the motor direction to these IN1 and IN2 pins on the H-bridge.
15. Put your linkage at the starting position, against the positive stop.
16. Make sure the power supply is set to 9V.
17. **Check your wiring connections before plugging in the power, also have a friend or teammate nearby. Never work alone when the power supply is plugged in.**
18. Plug in the power supply.
19. A green light on the power supply plug should turn on, and a red LED on the H-Bridge should light up. Contact your lab instructor if this doesn't happen as you may have a faulty power supply or H-Bridge

Now we will test the H-Bridge by seeing if it outputs an expected value.

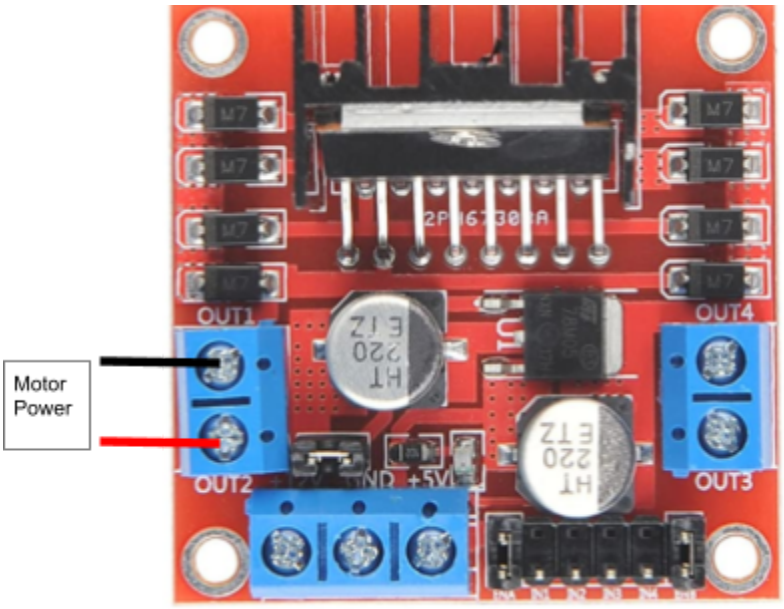
20. Download the "ME350_F22_HBridge_Test.ino".
21. Plug the Arduino into your computer and upload the sketch.
22. Turn the toggle switch ON.

23. Grab your DMM and set the dial to 20V for DC voltage. Touch the red probe to the OUT2 terminal and the black probe to the OUT1 terminal (reminder NOT the screw heads). You should see around 9V on your multimeter. NOTE: DO NOT let the metal probes touch each other while touching the terminals, since this will short the H-Bridge.



24. You are now done with the DMM and can set it aside.
25. **Turn off the toggle switch.**
26. **Unplug the power supply.**

27. Using the screw terminals on the H-Bridge, screw the crimped ferrules into the H-Bridge per the diagram below.
    a. Connect the red motor wire to “**OUT2**” and the black motor wire to “**OUT1**”



Exercise 3.2: Overcoming Friction

In this section, we will find the voltage needed to overcome friction in the mechanism. Keep your power supply plugged in.

1. Download the "ME350_W25_Generic_Target.ino" folder
    a. This code builds on the code from the Encoder test code from Exercise 2. **Read through the code to gain an understanding of what it does.**
2. Connect to your Arduino and plug in the power supply.
3. Make sure the toggle switch is OFF.
4. Open the ME350_W25_Generic_Target.ino sketch.
5. To start, make sure **FRICTION_COMP_VOLTAGE** = 0. Upload the code.
6. Open the serial monitor.
7. Now turn the toggle switch ON.
    a. Since your **FRICTION_COMP_VOLTAGE** value is 0, the linkage should not move.
8. Turn the toggle switch to the off position.
9. Increase the value of **FRICTION_COMP_VOLTAGE** slightly and re-upload the code.
    a. We recommend a 0.1V to 0.5 V increase each time.
    b. **FRICTION_COMP_VOLTAGE should always be a positive value.** If your motor goes in the negative direction, swap the power inputs (see the troubleshooting section).
10. Turn the toggle switch to the on position.
11. Repeat steps 8-10 while steadily incrementing **FRICTION_COMP_VOLTAGE**, re-uploading the code every time the value is changed, until the linkage barely moves.
    a. Once the linkage just barely moves, stop the process. This is your voltage needed to overcome friction in the mechanism.
    b. Record this value.
12. Once you have found your feedforward voltage, **turn off the toggle switch.**

13. Unplug the power supply and clean up.

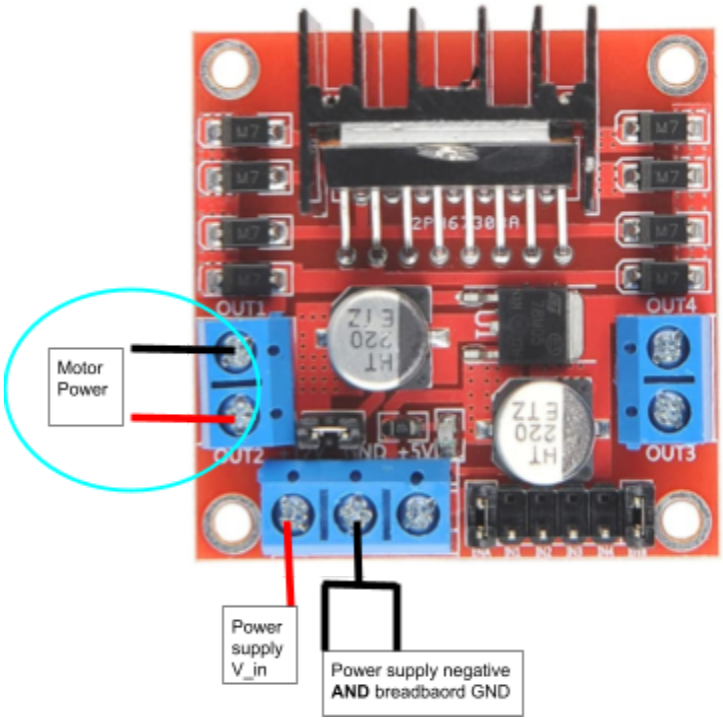
If working on a CAEN computer in the mechatronics lab, **save your sketch** to a flash drive or email it to yourself. Do not save your sketch to the computer in the mechatronics lab – it could be erased when reloading new software.

Troubleshooting

NOTE 1: If you want to cut power to the motor quickly, you have a few options:

1. Turn off the toggle switch
2. Remove the power supply from the wall plug
3. Disconnect the green screw terminal connector from the wall plug
4. Disconnect the bullet tip connector
5. Cut power to the wall plug power strip, if you are using one

NOTE 2: If your motor is running in the forward direction and the serial monitor is saying it is running in reverse, it means you have mounted your motor differently (for instance, upside down) compared to the prototype for which this Arduino sketch was written. No need to change your motor mounting. Just turn off the power supply, and switch these two wires going into the H-bridge (you can also swap the ends of the bullet tips connectors to achieve the same effect). This is because we always want **FRICTION_COMP_VOLTAGE** to be a positive value.



NOTE 3: If your motor is running in the correct direction to match the serial monitor, but the encoder is counting down when the motor is traveling forward, you may need to switch the white and yellow wires that are going from the motor into the breadboard.

NOTE 4: Pressing the white pushbutton on the Arduino board will restart the program. This is an easy way to reset the encoder counter to zero. Do this only after you have moved your mechanism back to the starting position.

Exercise 3: Wiring Diagram (Part of Lab 8 Assignment)

Make a wiring diagram of the work you've done in exercises 1-2. Save it for submission. You can feel free to use whatever tool you like for wiring diagrams, but here are some suggestions:

- TinkerCAD Circuits
- Google Sheets/PowerPoint
- Microsoft Visio
- Fritzing
- Hand-drawn Diagrams